Impedance BC Implementation Comparison

Prior vs new IMPEDANCE implementation and IMPEDANCE vs RCR behavior

Scope

- Solver code deltas in IMPEDANCE block setup and runtime behavior
- Functionality: convolution controls, memory persistence, and validation
- Rationale for major design decisions and tradeoffs
- Implications for 3D-0D coupling and full 0D simulation modes
- Evidence from fixtures and tests in svZeroDSolver/svZeroDTrees/svzt-agent

Comparison Axis A: Prior -> New IMPEDANCE

- Period ownership moved to simulation_parameters.cardiac_period
- Configure signature simplified; parser-level period requirement removed
- Error handling now tied to model dt/period consistency at initialization

Comparison Axis B: IMPEDANCE vs RCR

- History-based periodic convolution vs low-order lumped approximation
- Persistent memory semantics under coupling retries and commits
- Runtime cost/accuracy tradeoff via exact vs truncated kernel usage

Architecture Context

Where IMPEDANCE behavior differs in full 0D and 3D-0D workflows

Full 0D Simulation Path

JSON boundary_conditions + simulation_parameters

Model::update_time() -> ImpedanceBC::update_time()
Convolution uses accepted one-cycle history

- cardiac_period must align with dt and kernel length
- Kernel history starts after one accepted period
- Exact/truncated modes control per-step cost

3D-0D Coupled Iteration Path

Interface retry loop:
update_state() -> run_simulation() -> return_y()/return_ydot()

Committed persistent-state snapshot controls rollback-safe retries

- Deterministic trial reruns from identical committed state
- Commit occurs when caller accepts state via return_y/return_ydot
- Behavior explicitly verified in interface test_04

Solver Code Changes

Period ownership refactor and parser/configuration wiring updates

Prior implementation

- Required period inside bc_values for each IMPEDANCE BC
- Parser checked or set model.cardiac_cycle_period from BC field
- configure(z, period, pd, mode, num_kernel_terms)

New implementation

- period removed from IMPEDANCE bc_values contract
- Single source of truth: simulation_parameters.cardiac_period
- configure(z, pd, mode, num_kernel_terms)

Code-level evidence (HEAD commit 161870a5)

```
ImpedanceBC::configure signature
- old: configure(const std::vector<double>& z, double period, double pd, ...)
+ new: configure(const std::vector<double>& z, double pd, ...)

Initialization guard in ensure_initialized()
const double period = model->cardiac_cycle_period;
if (period <= 0.0) throw runtime_error(...cardiac_period > 0...);

SimulationParameters::create_boundary_conditions
period parsing and BC-level period consistency checks removed.
IMPEDANCE now configures from z, Pd, convolution_mode, num_kernel_terms.
```

Runtime Behavior of New IMPEDANCE BC

Discrete periodic convolution + one-cycle ring buffer memory

Olufsen Discrete Convolution

$$P_{n+1} = P_d + z_0 Q_{n+1} + \sum_{m=1}^{N_k - 1} z_m Q_{n+1-m}$$

Implicit term: z0*Q(n+1)
Added directly to system matrix (F).
Lagged terms use accepted flow history.

lagged lookup

One-cycle ring buffer

- head -> Q(n)
- Q(n-1)
- Q(n-2)
- ...
- Q(n-Np+1)

- Implicit term z0*Q(n+1) is assembled in F; lagged terms accumulate into C
- Lagged flows are read from accepted-state ring buffer (not transient trial state)
- Startup rule: lagged convolution disabled until one full accepted period elapsed
- Runtime checks enforce integer period/dt and z.size() == period steps

Execution modes

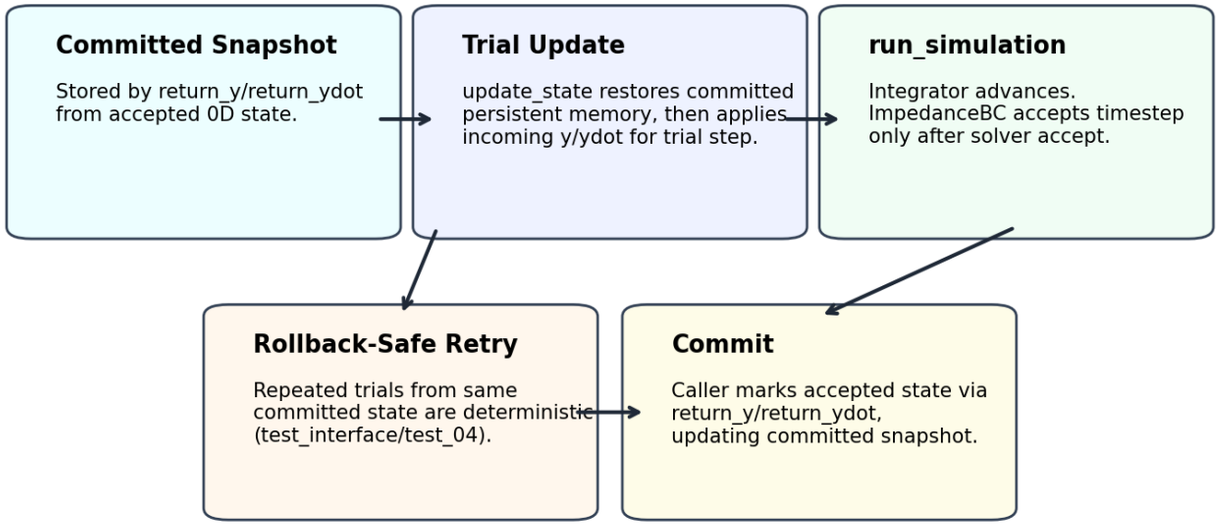
- exact: num_kernel_terms = Np (full period)
- truncated: user sets num_kernel_terms in [1, Np]
- Per-step convolution cost scales with num_kernel_terms

Key invariants

- dt must remain fixed after initialization
- Persistent state compatibility checks include kernel size
- Corrupted state payloads fail fast with explicit runtime errors

Coupling Trial/Accept Semantics

Rollback safety for persistent memory in repeated external-coupling trials



- return_y()/return_ydot() snapshots committed block persistent state
- update_state() restores committed state before each new trial
- run_simulation() advances from rollback-safe memory baseline

Evidence from tests

- test_interface/test_04 repeats trial runs from same committed state
- Asserts deterministic equivalence across repeated trials
- Asserts observable state change after explicit commit
- Confirms IMPEDANCE persistent memory participates in coupling protocol

Implication

3D solvers can probe multiple sub-iterations per external step without contaminating accepted impedance history.

Functionality and Validation Surface

What the new implementation enables and how it is guarded

Capability / Guard	Behavior	Failure Mode / Message
Kernel modes	exact (full Np) and truncated (k <= Np)	Invalid mode or k out of range throws runtime_error
Period source	simulation_parameters.cardiac_period only	cardiac_period <= 0 rejected at initialization
Temporal consistency	Requires integer period/dt and fixed dt post-init	Non-integer ratio or dt change rejected
Kernel shape	z must be finite array with z.size() = Np	Missing/invalid/non-finite kernel rejected
Persistent state integrity	Supports get/set with compatibility checks	Corrupted payload or size mismatch rejected

- Compared with the prior contract, validation is centralized around model-level temporal consistency.
- This removes duplicated period declarations and reduces mismatch states between BCs and global simulation settings.

Implementation Decision Rationale

Why these design choices were made and what they trade off

Decision	Justification	Tradeoff
Remove BC-level period field	Single source of truth in simulation_parameters improves consistency and parser simplicity.	Requires users to set global cardiac_period explicitly for IMPEDANCE.
Initialize from model dt and cardiac_period	Runtime checks align kernel discretization with actual integration time-step.	Less tolerant to loosely specified inputs (fails fast by design).
Persistent state snapshot/restore in interface	Guarantees deterministic trial reruns for 3D coupling sub-iterations.	Adds interface bookkeeping for committed block states.
Exact + truncated convolution modes	Exposes accuracy/performance knob for large kernels and long periods.	Users must choose num_kernel_terms carefully for truncated mode.

IMPEDANCE vs RCR

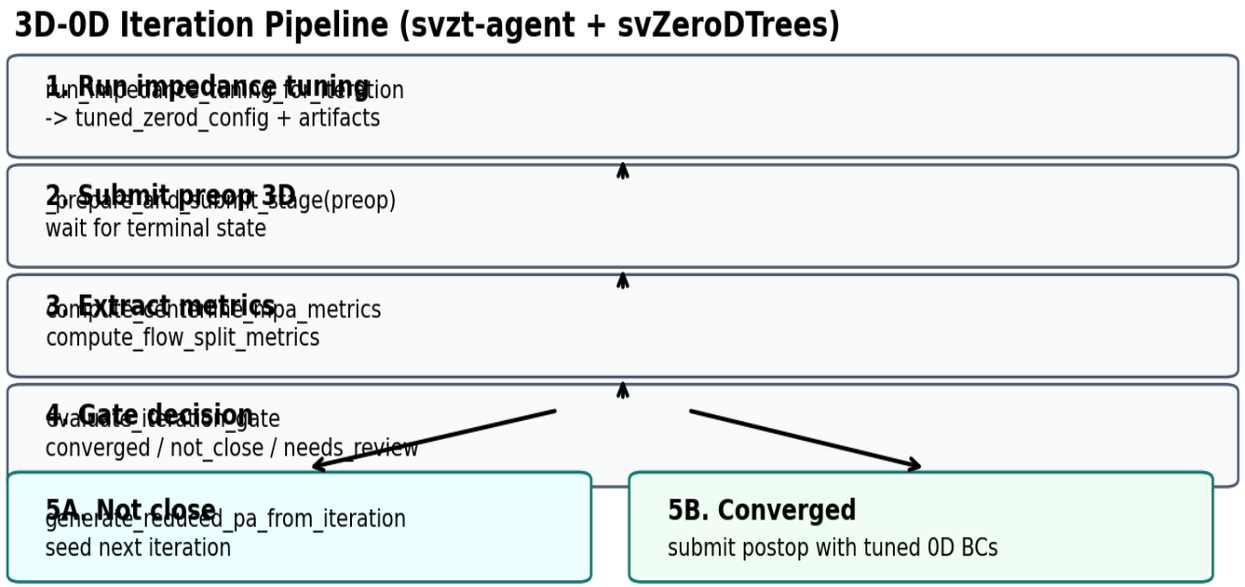
Behavioral and practical comparison for model selection

Dimension	IMPEDANCE BC	RCR BC
Memory	One-cycle flow history ring buffer; explicit persistent state	Low-order internal states only; no long kernel memory
Frequency content	Directly encodes structured-tree kernel waveform	Approximates with 3-parameter lumped dynamics
Coupling retries	Designed for trial/accept rollback-safe semantics	Also retry-safe, but without long convolution history
Runtime cost	O(num_kernel_terms) per accepted step	Constant-time low-order update
Input contract	z kernel + global cardiac_period + mode	Rp, C, Rd parameters (+ optional variants)
Use cases	High-fidelity distal bed effects / wave reflections	Fast reduced-order approximation and robust baseline

Practical interpretation: IMPEDANCE provides richer distal dynamics at higher runtime/config complexity; RCR remains the lightweight baseline.

3D-0D Workflow Implications

How the new impedance path behaves inside iterative tuning/submission loops



- Tuning artifacts: optimized_params.csv, PA snapshot, tuned 0D coupling config
- Gate outputs drive branch: converged -> postop submit, else regenerate reduced PA seed

Operational implications

- Impedance tune_space is now validated end-to-end across defaults and patient overrides
- Decision payloads explicitly include convergence state and regeneration path
- 3D submission logic can pause to needs_review on missing artifacts or failed extraction
- Pipeline naturally separates physics tuning from scheduler lifecycle handling

Relevant modules

```
svzerodtrees.tuning.iteration
svzt-agent templates/slurm/job_template.sh
svzt config model/load merge rules
```

Full 0D Implications

Configuration contract and reproducibility behavior after the refactor

Input contract update

```
// old (removed)
"bc_values": { "period": 1.0, "Pd": 80000.0, "z": [...] }

// new
"simulation_parameters": { "cardiac_period": 1.0, ... }
"bc_values": { "Pd": 80000.0, "z": [...], "convolution_mode":
"exact" }
```

- Cardiac period is globally defined and consumed by IMPEDANCE at initialization
- Kernel length and period/dt mismatch now fail fast with explicit diagnostics
- Fixture updates moved period declarations from BC block to simulation parameters

Reproducibility and debugging

- Deterministic behavior depends on fixed dt and explicit committed-state transitions
- Persistent state serialization includes dt, period steps, kernel terms, and flow history
- Corrupted or incompatible persistent payloads trigger guarded runtime failures
- Error messages now reference missing/invalid global cardiac_period rather than BC period

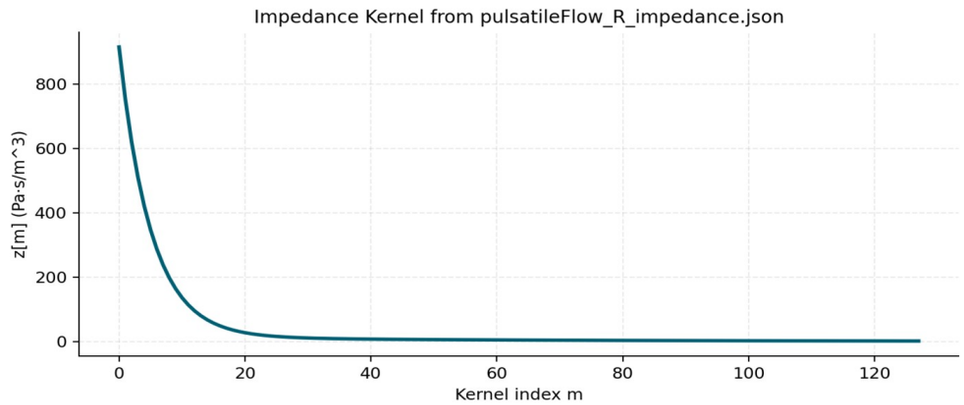
Recommended full-0D checklist

- Set simulation_parameters.cardiac_period
- Ensure period/dt is an integer count
- Set z length to one period in time steps
- Use truncated mode only with validated num_kernel_terms

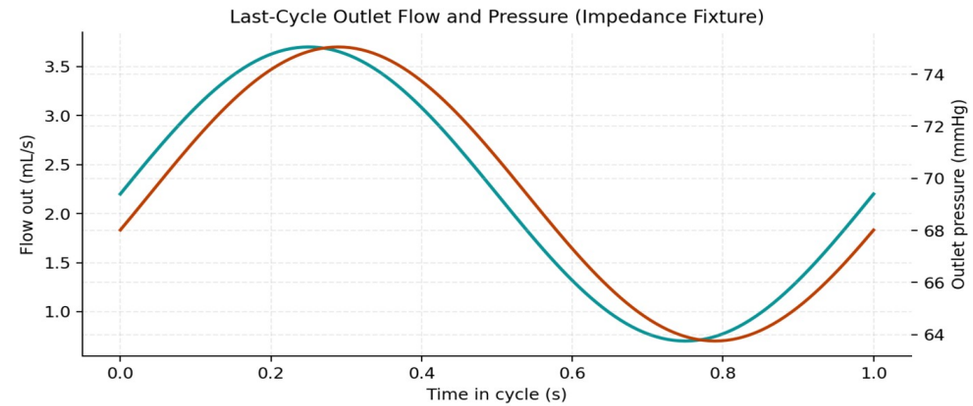
Evidence from Fixtures and Tests

Kernel and waveform evidence plus explicit test coverage

Kernel fixture evidence



Flow/pressure fixture evidence



Key tests and fixtures referenced

- tests/cases/pulsatileFlow_R_impedance.json and result_pulsatileFlow_R_impedance.json
- tests/test_interface/test_04 (coupling trial/accept deterministic semantics)
- src/model/ImpedanceBC.{h,cpp} and src/solve/SimulationParameters.cpp
- svzerodtrees/tuning/iteration.py and svzt-agent templates/slurm/job_template.sh

Takeaway: the refactor improves contract consistency and coupling safety while preserving a clear runtime cost/accuracy control surface.